Fluxo de Desenvolvimento do Painel BMAD

Especificação funcional do processo de desenvolvimento assistido por IA

Versão consolidada - Setembro de 2026

1. Visão geral

Objetivo. O painel deverá transformar o BMAD num processo de desenvolvimento guiado, compreensível para utilizadores não técnicos e suficientemente aberto para programadores experientes. O BMAD funciona como motor de execução; o painel controla estado, decisões, contexto, qualidade, segurança, rastreabilidade e aprendizagem do projeto.

Fluxo principal: Ideia → Descoberta → Product Brief/Brainstorm → PRD → UX → Arquitetura → Epics → Stories → Desenvolvimento → Validação → Entrega.

Princípio central. O utilizador não deverá precisar de conhecer agentes, comandos internos ou a metodologia BMAD. O painel traduz intenções de negócio para workflows técnicos e devolve decisões e resultados numa linguagem adequada ao perfil do utilizador.

2. Modos de utilização

O nível de intervenção do utilizador deverá ser uma configuração transversal ao projeto, e não apenas uma opção da fase de arquitetura.

Modo	Comportamento	Utilizador-alvo
Guiado	O sistema recomenda opções, reduz decisões técnicas e solicita apenas confirmações relevantes.	Utilizador leigo ou de negócio
Colaborativo	A IA recomenda, mas o utilizador participa das principais decisões funcionais e técnicas.	Product owner, analista ou utilizador com conhecimento intermédio
Especialista	Expõe escolhas técnicas, metodologias e configurações avançadas.	Programador, arquiteto ou equipa técnica

3. Modelo padrão de cada fase

Cada etapa do processo deverá possuir uma estrutura previsível para permitir automatização, auditoria e controlo de avanço.

- Objetivo da fase.
- Entradas obrigatórias.
- Workflow ou agente responsável.
- Artefactos produzidos.
- Revisões automáticas ou humanas.
- Critérios de saída (gate).
- Dependências e decisões registadas.

4. Fases do desenvolvimento

4.1 Ideia

Fase anterior aos workflows nativos do BMAD. O objetivo é transformar uma intenção vaga num problema de negócio minimamente definido.

- Problema que se pretende resolver.
- Público ou utilizadores afetados.
- Resultado esperado.
- Tipo de produto: aplicação interna, SaaS, marketplace, app móvel, automação, portal, entre outros.
- Restrições iniciais de prazo, custo, equipa, regulação ou operação.

Gate de saída — Ideia pronta para Descoberta

- Problema e público identificados.
- Resultado esperado descrito.
- Restrições críticas conhecidas ou marcadas como pendentes.

4.2 Descoberta

Fase sistematizada com apoio de gestão/projetos. A intenção é dar ao cliente visão de negócio antes de entrar no aprofundamento do BMAD.

- Personas e utilizadores.
- Problemas, dores e necessidades.
- Jobs to be done e fluxos atuais.
- Hipóteses de solução.
- Riscos de negócio.
- Modelo de receita, quando aplicável.
- Dependências externas e integrações.

- Restrições legais, operacionais ou de segurança já conhecidas.

Artefacto de saída: Discovery Brief.

Gate de saída — Descoberta concluída

- Discovery Brief gerado.
- Principais hipóteses e riscos registados.
- Informação suficiente para iniciar Brainstorm sem depender de conhecimento implícito do cliente.

4.3 Product Brief / Brainstorm

O BMAD possui várias técnicas de brainstorm. O painel deverá reduzir a carga cognitiva do utilizador e evitar que a escolha de uma metodologia inadequada torne o processo excessivamente complexo.

- No modo Guiado, o sistema escolhe automaticamente as técnicas adequadas com base no Discovery Brief.
- No modo Colaborativo, apresenta poucas técnicas recomendadas, acompanhadas de explicação simples.
- No modo Especialista, permite acesso ao conjunto mais amplo de ferramentas do BMAD.
- As perguntas devem ser adaptadas ao nível de conhecimento do utilizador; perguntas excessivamente técnicas deverão ser traduzidas ou inferidas a partir do contexto existente.

Artefactos de saída: Brainstorm consolidado e Product Brief.

Gate de saída — Product Brief pronto

- Problema, oportunidade e proposta de valor suficientemente desenvolvidos.
- Hipóteses e funcionalidades iniciais registadas.
- Contexto suficiente para produção do PRD.

4.4 PRD

O PRD pode ser produzido colaborativamente ou de forma amplamente autónoma quando o Brainstorm/Product Brief contém contexto suficiente.

- Requisitos funcionais e não funcionais.
- Objetivos e métricas de sucesso.
- Escopo e não-escopo.
- Personas e cenários relevantes.
- Regras de negócio.
- Dependências, integrações e restrições.
- Critérios de aceitação em nível de produto.

Gate de saída — PRD aprovado

- Requisitos essenciais cobertos.
- Contradições relevantes resolvidas.
- Escopo suficientemente claro para orientar UX e arquitetura.

4.5 UX

A UX deverá ocorrer antes da arquitetura porque decisões de experiência e fluxos podem alterar a estrutura técnica necessária. O BMAD já conduz esta etapa de forma eficiente, portanto a intervenção do painel deve ser mínima e focada em apresentação e estado.

- Fluxos principais e alternativos.
- Navegação e hierarquia de informação.
- Estados de interface, erros e exceções.
- Responsividade e acessibilidade.
- Referências visuais e componentes quando aplicável.

Artefactos de saída: Especificação UX, fluxos e referências necessárias para arquitetura e stories.

4.6 Arquitetura

Esta fase deverá combinar recomendação automática com capacidade de decisão. Para utilizadores leigos, o painel não deverá começar por perguntas técnicas isoladas como “monólito ou microserviços?”, mas sim apresentar uma recomendação fundamentada e permitir personalização.

4.6.1 Seleção assistida da arquitetura

- O sistema analisa Discovery, Product Brief, PRD e UX.
- Apresenta uma arquitetura recomendada com justificativa simples.
- O utilizador pode aceitar ou abrir alternativas.
- Cada alternativa deve conter explicação breve: quando se aplica, vantagens, limitações e impacto operacional.

4.6.2 Stack tecnológica

Quando houver escolha manual, as opções deverão ser limitadas e encadeadas para evitar combinações incoerentes.

- Backend: PHP, C#, Java, Python e outras tecnologias explicitamente suportadas.
- Framework de acordo com a linguagem: Laravel, .NET, Spring, Django/FastAPI, etc.
- Frontend e aplicações móveis conforme o tipo de produto.
- Base de dados: relacional/SQL ou NoSQL, seguida da tecnologia concreta.
- Cache, filas, pesquisa, storage e serviços auxiliares quando necessários.

4.6.3 Infraestrutura e deploy

A estratégia de entrega deve nascer juntamente com a arquitetura e não ser deixada para o final do projeto.

- Cloud provider ou ambiente de alojamento.
- Containers e estratégia de execução.
- CI/CD.
- Ambientes dev, staging e produção.
- Gestão de secrets.
- Base de dados, storage, CDN e rede.
- Observabilidade, logs e alertas.
- Backups e rollback.

4.6.4 Architecture Decision Records (ADRs)

Cada decisão arquitetónica relevante deverá gerar um registo com decisão, motivos, alternativas avaliadas e consequências. Os ADRs tornam as escolhas rastreáveis e evitam alterações silenciosas feitas por agentes durante o desenvolvimento.

Gate de saída — Arquitetura pronta

- Arquitetura e stack definidas.
- Infraestrutura e deploy definidos em nível suficiente para iniciar implementação.
- Baseline de segurança criada.
- Principais ADRs registados.

4.7 Epics

A criação de Epics pode continuar maioritariamente autónoma através do BMAD. O painel deve verificar consistência com PRD, UX e arquitetura e organizar os Epics como unidades de entrega.

Regra operacional inicial: por defeito, cada Epic constitui um ciclo de entrega equivalente a uma Sprint. A arquitetura do produto deve, porém, permitir que futuramente um Epic contenha mais de uma Sprint.

4.8 Stories

Cada Story passa por um pipeline formal antes de poder entrar em desenvolvimento.

1. /bmad-create-story — geração da Story.
2. Revisão adversarial — identificação de ambiguidades, conflitos, lacunas e interpretações divergentes.
3. Correção dos achados da revisão adversarial.
4. Revisão de segurança — análise dos requisitos da Story antes da implementação.
5. Correção dos achados de segurança na especificação.
6. Story Readiness Check — validação final de prontidão.

Story Readiness Check

- Critérios de aceitação claros e testáveis.
- Dependências identificadas.

- Arquitetura e ADRs respeitados.
- UX referenciada quando aplicável.
- Requisitos de segurança incorporados.
- Testes esperados descritos.
- Ausência de ambiguidades ou TODOs bloqueantes.

Estado resultante: READY FOR DEV.

4.9 Desenvolvimento

Uma Story só pode ser marcada como concluída após implementação, revisão, segurança e verificação automatizada.

7. /bmad-dev-story — implementação da Story.
8. Execução de testes automatizados relevantes.
9. /bmad-code-review — revisão de código.
10. Correção dos achados do code review.
11. Revisão de segurança pós-implementação.
12. Correção dos achados de segurança.
13. Verificação final automatizada.

Gate de conclusão da Story

- Build concluído.
- Testes obrigatórios aprovados.
- Code review sem achados bloqueantes.
- Security review sem achados bloqueantes.
- Critérios de aceitação atendidos.
- Migrações, contratos e dependências validados quando aplicável.

Estado resultante: DONE.

4.10 Validação do Epic

Ao final de cada Epic/ciclo de entrega será executada uma validação integrada antes da release. A validação deverá cobrir três dimensões.

Dimensão	Verificações principais
Técnica	Build, testes, integração, segurança, arquitetura, performance e qualidade do código.
Produto	Conformidade com PRD, regras de negócio, requisitos e critérios de aceitação.
UX	Fluxos, estados, responsividade, acessibilidade

	e aderência às decisões de UX.

4.11 Retrospectiva e aprendizagem do projeto

Após a validação do Epic, o workflow de retrospectiva do BMAD deverá ser executado. O resultado não será apenas histórico: deverá alimentar o comportamento dos agentes nos Epics seguintes.

- Project Learnings — contexto, problemas recorrentes e conclusões.
- New Guardrails — regras objetivas derivadas da experiência do Epic.
- Atualização da memória controlada do projeto.
- Atualização de padrões, ADRs ou convenções quando necessário.

Exemplo: se uma retrospectiva identificar três implementações diferentes para tratamento de erros, poderá criar um guardrail exigindo que todas as APIs usem o mesmo componente ou padrão definido pelo projeto.

4.12 Entrega

Com o Epic validado e a retrospectiva processada, o ciclo pode ser disponibilizado no ambiente de destino definido pela arquitetura.

- Release preparada e identificada.
- Deploy conforme pipeline definido.
- Verificações pós-deploy.
- Registo de versão e funcionalidades entregues.
- Rollback disponível quando aplicável.
- Rastreabilidade dos requisitos incluídos na release.

Gate de entrega

- Validação técnica, produto e UX aprovada.
- Achados bloqueantes resolvidos.
- Deploy executado ou pacote de entrega preparado.
- Release registada e rastreável.

5. Rastreabilidade

A plataforma deverá manter relações explícitas entre artefactos para permitir saber de onde cada funcionalidade surgiu e onde foi implementada.

Discovery → PRD Requirement → UX Flow → Epic → Story → Código/Commit → Teste → Release

- Cada requisito relevante deve possuir identificador estável.
- Stories devem apontar para requisitos e fluxos UX relacionados.
- Resultados de testes e validações devem apontar para Stories.
- Releases devem listar os requisitos e Stories incluídos.

6. Project Constitution e memória controlada

Os agentes não deverão receber indiscriminadamente todo o histórico do projeto. O painel deve manter uma memória resumida e controlada contendo regras duradouras.

- Arquitetura e stack.
- ADRs vigentes.
- Naming conventions e padrões de código.
- Baseline e regras de segurança.
- Guardrails derivados de retrospectivas.
- Dependências autorizadas e restrições.
- Práticas proibidas ou decisões já descartadas.

Princípio de contexto: cada agente recebe Project Constitution + contexto mínimo necessário para executar a tarefa atual.

7. Segurança como gate transversal

A segurança participa do processo em dois momentos principais: antes da implementação e depois do desenvolvimento. O relatório deve ser compreensível, rastreável e acionável.

- Classificação do achado: baixo, médio ou alto.
- Descrição do problema.
- Explicação do risco.
- Recomendação de correção.
- Estado da correção.
- Ligação ao artefacto analisado: Story, código, arquitetura ou release.

Achados definidos como bloqueantes pela política do projeto impedem a progressão para o próximo gate até serem corrigidos ou formalmente tratados conforme a governança definida.

8. Princípios de UX do painel

A interface principal deverá representar o estado do desenvolvimento e não a estrutura interna de agentes do BMAD.

- Mostrar fases como “Definindo o produto”, “Preparando experiência”, “Definindo tecnologia”, “Desenvolvendo” e “Validando”.
- Ocultar comandos BMAD no modo Guiado.
- Expor detalhes técnicos e agentes apenas quando fizer sentido para perfis avançados.
- Apresentar recomendações acompanhadas de justificativa curta e opção de personalização.
- Destacar gates, pendências, decisões e bloqueios.
- Permitir visualizar histórico e rastreabilidade sem obrigar o utilizador a ler logs de agentes.

9. Fluxo consolidado

IDEIA

↓

DESCOBERTA

↓

DISCOVERY BRIEF

↓

BRAINSTORM / PRODUCT BRIEF

↓

PRD

↓

UX

↓

ARQUITETURA

- Stack
- Infraestrutura e Deploy
- Security Baseline
- ADRs

↓

EPICS

↓

CICLO DO EPIC

Story
↓ Adversarial Review
↓ Correções
↓ Security Review
↓ Correções
↓ Story Readiness
↓ READY FOR DEV

Desenvolvimento
↓ Testes
↓ Code Review
↓ Correções
↓ Security Review
↓ Correções
↓ Verification
↓ DONE

Repetir para as Stories restantes

↓

VALIDAÇÃO DO EPIC

• Técnica
• Produto
• UX

↓

RETROSPECTIVA

• Project Learnings
• New Guardrails

↓

RELEASE / ENTREGA

↓

PRÓXIMO EPIC

10. Princípios estruturais do produto

- BMAD é o motor; o painel é a camada de governança e experiência.
- O utilizador escolhe o nível de autonomia e detalhe técnico.
- Nenhuma Story entra em desenvolvimento sem passar pelos gates definidos.
- Nenhuma Story é concluída apenas porque o agente gerou código.
- Segurança é validada antes e depois da implementação.
- Arquitetura e deploy são decisões de início de projeto, não de finalização.
- Retrospectivas geram aprendizagem operacional e novos guardrails.
- Decisões relevantes devem ser rastreáveis e persistentes.
- Agentes recebem contexto mínimo necessário, reduzindo ruído e deriva do projeto.

Resultado esperado: uma plataforma de desenvolvimento assistido por IA na qual velocidade de execução não elimina governação, controlo técnico, segurança, rastreabilidade nem capacidade de decisão humana.